

Report finale (tiling)

Ghidoni Gaia

1 Introduzione e overview

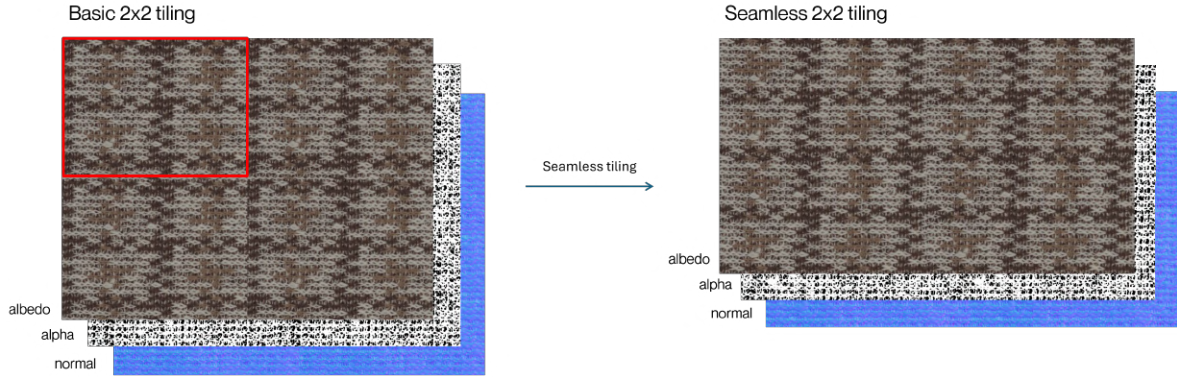


Fig. 1: Input e output della procedura di tiling su tessuti digitalizzati.

La ripetizione di una piccola immagine di texture su una superficie estesa è una tecnica comunemente utilizzata nella grafica digitale e nella modellazione dei materiali per simulare l'aspetto di materiali continui. Affinché tale ripetizione risulti visivamente coerente, la texture di partenza deve essere priva di discontinuità ai bordi, ovvero *seamless* (Fig. 1). Nel contesto della digitalizzazione dei tessuti, ottenere texture seamless a partire da immagini reali rappresenta una sfida significativa, in particolare quando si richiede una rappresentazione accurata e consistente delle proprietà fisiche del materiale.

A tal fine è stato sviluppato il software STT (Seamless Tiling Tool), progettato per generare automaticamente texture ripetibili a partire da immagini PBR (Physically Based Rendering). Il software opera simultaneamente su tutte le mappe PBR associate, garantendo la coerenza tra colore, normal, roughness e le altre proprietà del materiale. Il processo prevede una fase di interazione utente per la selezione di una regione che contenga o approssimi il modulo del tessuto, seguita dall'identificazione automatica del modulo ripetibile mediante il confronto dei bordi tramite una *sliding window* (Fig. 2). Il modulo individuato viene quindi memorizzato e utilizzato per generare e visualizzare un tiling 3×3 , permettendo una verifica immediata della continuità e della qualità della texture risultante.

Non tutti i pattern possono essere ricondotti a un modulo rettangolare. Per gestire correttamente pattern che presentano periodicità lungo assi non ortogonali, il software consente all'utente di selezionare una regione di forma parallelogramma (Fig. 3). A

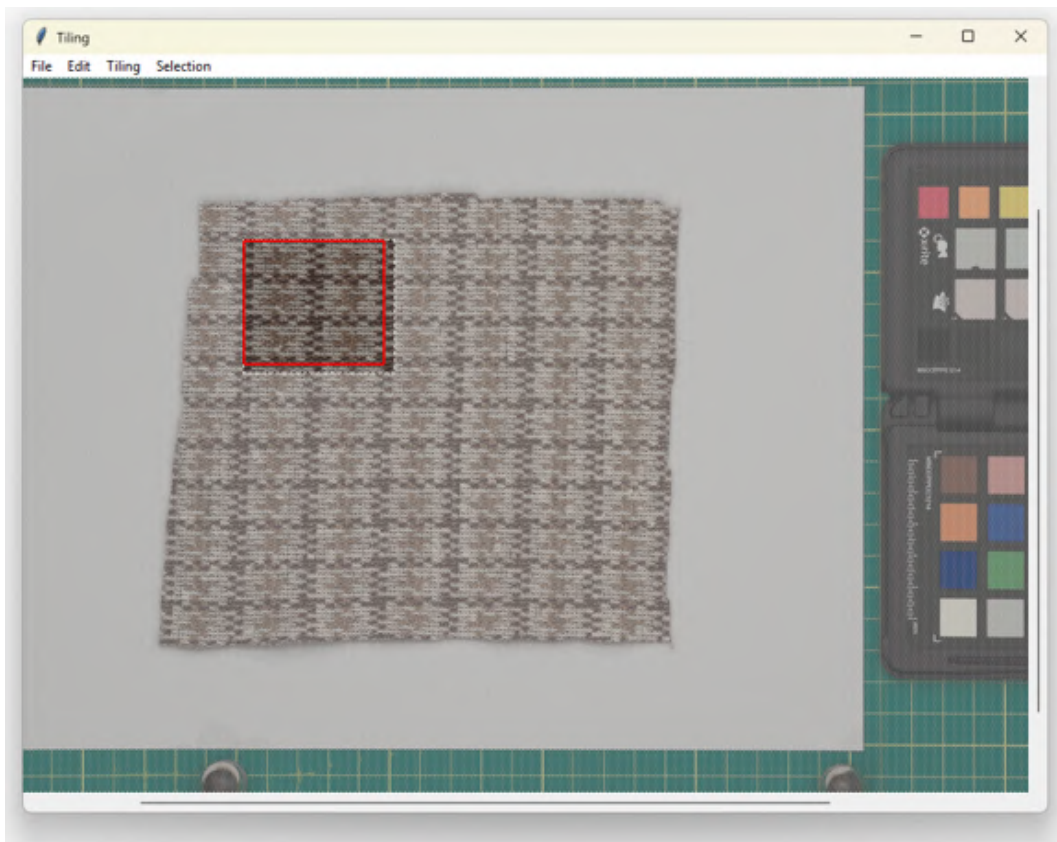


Fig. 2: Software STT: risultato dell'individuazione del modulo (in rosso), rispetto all'area selezionata dall'utente (tratteggio bianco).

partire da tale selezione, il modulo ripetibile viene individuato seguendo la stessa procedura adottata per il caso rettangolare, permettendo così la generazione di un seamless tiling anche per strutture con simmetrie non ortogonali.

Texture di tipo organico o caratterizzate da deformazioni locali possono, generare artefatti visivi durante il processo di tiling. Per mitigare tali problematiche, è stato integrato nel software STT uno strumento basato su Graph Cut (GC) [2], che utilizza un algoritmo di min-cut/max-flow [1, 4] per determinare transizioni ottimali tra le regioni adiacenti della texture (Fig. 4b). Questo approccio, chiamato *subpatching*, consente di individuare percorsi di taglio a minima discontinuità in una regione selezionata all'interno dell'immagine, migliorando la qualità visiva del tessuto digitalizzato. L'algoritmo opera simultaneamente su tutte le mappe PBR associate, assicurando la continuità e la consistenza delle proprietà fisiche del materiale.

Un ulteriore problema legato alle texture di tipo organico è la difficoltà nell'individuare un modulo ripetibile, poiché questi pattern spesso presentano caratteristiche altamente casuali. Per risolvere il problema del seamless tiling anche in questo scenario, è stato sviluppato un approccio basato su Graph Cut, che consente di modificare i bordi di un modulo inizialmente approssimativo, rendendolo perfettamente seamless (Fig. 4a). Questo metodo è stato testato con successo, ma rimane da eseguire l'integrazione nel software.

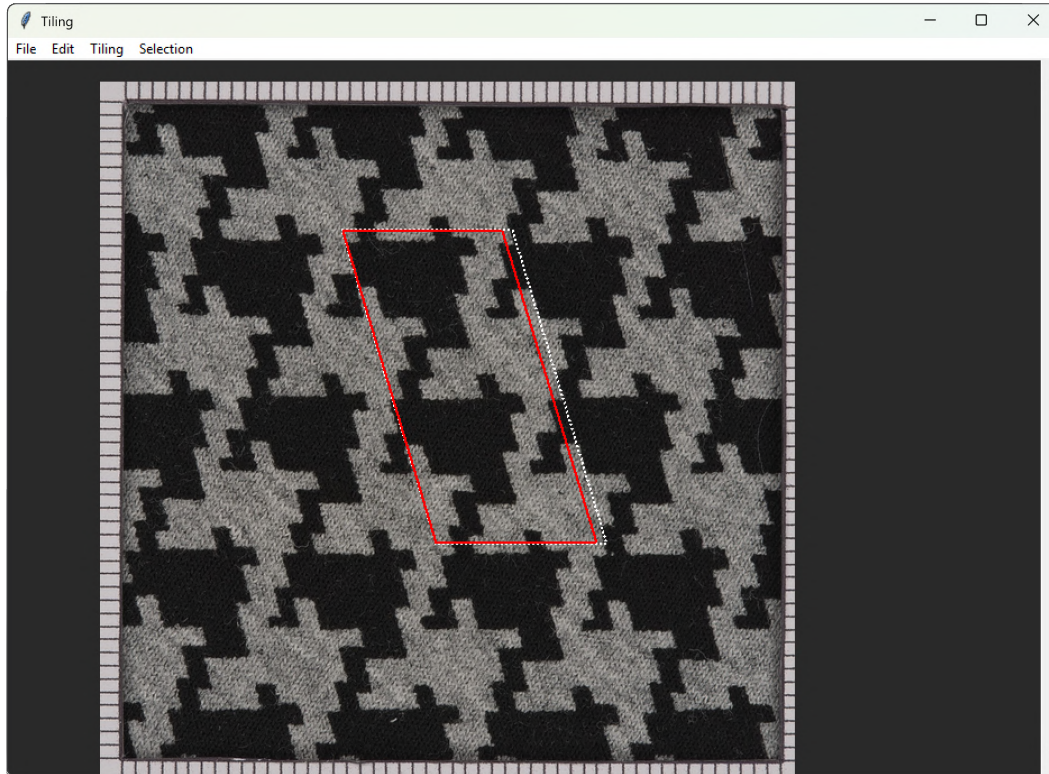


Fig. 3: Ricerca di un modulo parallelogramma: risultato dell'individuazione del modulo (in rosso), rispetto all'area selezionata dall'utente (tratteggio bianco).

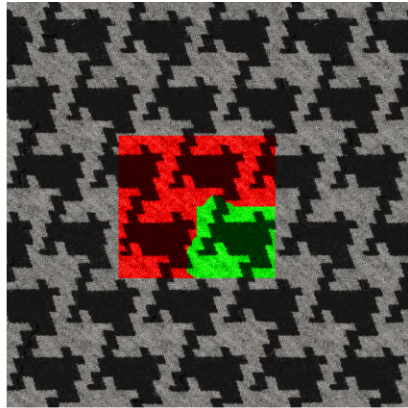
2 Architettura del software

2.1 Linguaggi e tecnologie utilizzate

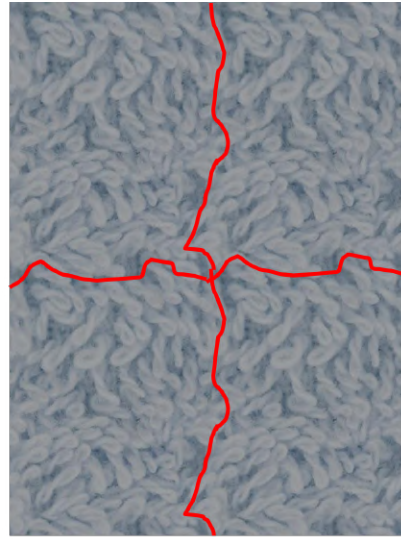
Il software è sviluppato in Python (versione 3.12.7). La scelta del linguaggio Python è motivata dalla natura del problema affrontato - l'elaborazione di immagini raster. Python mette a disposizione un ecosistema maturo di librerie per il calcolo numerico e l'immagine processing, consentendo una rapida prototipazione degli algoritmi e una chiara rappresentazione delle operazioni matematiche coinvolte.

Le principali tecnologie utilizzate sono:

- **NumPy** e **SciPy** per il calcolo numerico e la manipolazione di array multidimensionali;
- **Pillow** per il caricamento, la conversione e il salvataggio dei file immagine;
- **tkinter** e **skimage** per lo sviluppo dell'interfaccia grafica e l'interazione con l'utente;
- **Matplotlib** per la visualizzazione dei risultati durante le fasi di sviluppo e debugging;
- **NetworkX** per la modellazione dei grafi e l'applicazione degli algoritmi di min-cut/max-flow utilizzati nella fase di Graph Cut.



(a)



(b)

Fig. 4: Applicazione del Graph Cut (GC). (a) metodo del *subpatch*: un'area selezionata viene aggiornata con una patch (verde) ritagliata con GC. (b) un modulo organico è reso *seamless* grazie all'applicazione del GC.

2.2 Struttura del software

Il software STT è organizzato come un'applicazione standalone, strutturata in modo modulare. L'organizzazione dei file e delle directory riflette le diverse funzionalità del sistema e le fasi della pipeline di elaborazione.

La struttura principale del progetto è la seguente:

- **main.py**: punto di ingresso dell'applicazione. Gestisce l'avvio del software, l'inizializzazione dell'interfaccia grafica e il coordinamento delle principali fasi di elaborazione;
- **CanvasImage.py**: contiene i moduli relativi all'interfaccia grafica e alla gestione dell'interazione con l'utente. In particolare, implementa una classe basata su `Tkinter Canvas`, progettata per visualizzare immagini ingrandibili e navigabili, consentendo all'utente di selezionare liberamente regioni di interesse all'interno dell'immagine in esame;
- **Selection.py**: classe ausiliaria per la gestione grafica e numerica delle selezioni effettuate dall'utente sull'immagine. Gestisce la visualizzazione della selezione e il calcolo delle coordinate associate, incluse le conversioni tra il sistema di coordinate dello schermo e quello dell'immagine;
- **Rectangle.py** e **ShearRectangle.py**: classi ausiliarie derivate dalla classe **Selection**, che implementano le funzionalità specifiche per i due tipi di selezione considerati: selezioni rettangolari e selezioni a parallelogramma. In particolare, la sottoclasse **ShearRectangle** consente all'utente di modificare liberamente l'inclinazione del parallelogramma;
- **Tiling.py**: contiene la classe responsabile dell'esecuzione del tiling di base, ovvero la ricerca del migliore modulo di ripetizione della texture in esame;

- `Subpatch.py`: contiene la classe responsabile dell'esecuzione del subpatching e della gestione dei dati associati, coordinando i metodi e le funzioni basate su Graph Cut;
- `patch_fitting.py`: raccoglie i metodi dedicati alle procedure basate su Graph Cut utilizzate per il subpatching. Il modulo include la ricerca del patch ottimale per la sostituzione di una regione dell'immagine, il calcolo delle funzioni di costo, la costruzione del grafo e l'applicazione degli algoritmi di min-cut/max-flow;
- `utils.py`: contiene due classi ausiliarie per la gestione delle coordinate delle selezioni utente;
- `settings.py`: contiene alcune variabili globali utili alla gestione dell'interfaccia grafica;
- `img/`: directory contenente le immagini di input, costituite dalle mappe PBR dei tessuti da processare;
- `out/`: directory destinata al salvataggio dei risultati prodotti dal software;
- `out/subpatch_out`: directory destinata al salvataggio dei risultati di subpatching;
- `requirements.txt`: elenco delle dipendenze software necessarie per l'esecuzione del programma.

2.3 Modello dei dati

Il software opera su immagini raster. I dati sono costituiti da file di input forniti dall'utente e da file di output generati al termine delle elaborazioni.

2.3.1 Dati in ingresso

I dati di ingresso del software consistono in **una o più immagini PBR** associate a un materiale tessile. In particolare, sono supportate mappe quali albedo (colore), normal, roughness e, opzionalmente, altre mappe descrittive delle proprietà fisiche del materiale. Le immagini sono state testate nei formati PNG e JPEG, che rappresentano i formati di input attualmente supportati dal software.

Si assume che tutte le mappe PBR di input:

- abbiano la stessa risoluzione e dimensione;
- siano correttamente allineate e corrispondenti pixel a pixel;
- utilizzino un formato immagine supportato dal software.

In vincolo sulla risoluzione e dimensione delle mappe dei materiali può essere considerato un vincolo stretto: deve essere soddisfatto affinché il software possa eseguire correttamente l'algoritmo di ricerca. L'allineamento delle mappe PBR è una caratteristica fortemente consigliata. L'algoritmo non esegue controlli espliciti su tale aspetto; tuttavia, una mancata corrispondenza pixel a pixel tra le mappe può compromettere la qualità dei risultati della fase di individuazione del modulo.

Alle immagini PBR può essere associato un **file di testo contenente i pesi** relativi a ciascuna mappa. Tali pesi sono utilizzati dall'algoritmo di ricerca del modulo di ripetizione per bilanciare l'influenza delle diverse mappe.

Il file può avere un nome arbitrario, purché con estensione `.txt`, e deve contenere una sequenza di valori numerici separati da punto e virgola. I pesi sono elencati secondo l'ordine alfabetico dei nomi dei file immagine corrispondenti alle mappe PBR fornite in input. Ciascun valore rappresenta il peso assegnato alla rispettiva mappa nelle procedure di ricerca del modulo di ripetizione.

Ad esempio, nel caso di tre mappe denominate `albedo`, `normal` e `roughness`, i pesi possono essere specificati nel file di testo come:

1; 10; 1

2.3.2 Dati in uscita

I dati di uscita del software sono immagini raster generate a seguito del processo di elaborazione. In particolare, il software produce:

- il modulo di ripetizione individuato per ciascuna mappa PBR;
- immagini di anteprima, come il tiling 3×3 , utilizzate per la verifica visiva della continuità della texture.
- la texture modificata dalla procedura di subpatching.

Tutte le immagini di output mantengono la coerenza tra le diverse mappe PBR associate al materiale, garantendo una rappresentazione consistente delle proprietà fisiche del tessuto digitalizzato.

3 Algoritmi e metodi

Di seguito vengono descritti gli algoritmi e i metodi adottati dal software STT per l'ottenimento di texture seamless a partire da immagini PBR. Gli approcci presentati coprono sia la ricerca automatica del modulo di ripetizione sia le tecniche di ottimizzazione basate su Graph Cut, utilizzate per migliorare la continuità visiva delle texture in presenza di pattern complessi o irregolari.

Gli algoritmi operano su tutte le mappe PBR associate al materiale in modo coerente. In particolare, viene descritto il metodo di **ricerca del modulo** di ripetizione e l'impiego di algoritmi di min-cut/max-flow per le operazioni di **subpatching** e per il **miglioramento del seamless tiling**.

3.1 Ricerca del modulo

L'algoritmo di ricerca del modulo di ripetizione si basa sul confronto dei bordi della texture in un intorno dell'area selezionata dall'utente. Si assume che l'utente selezioni una ripetizione approssimata del pattern del tessuto, che viene utilizzata come riferimento per individuare automaticamente un modulo che possa essere ripetuto senza introdurre discontinuità visive ai bordi.

A partire dalla selezione iniziale, vengono individuate due coppie di **bande di confronto interne** alla regione: una orizzontale superiore (T) e una verticale sinistra (L), assunte come fisse e rappresentative della prima banda interna del modulo di ripetizione (Fig. 5). Un modulo è considerato *seamless* quando queste bande interne coincidono

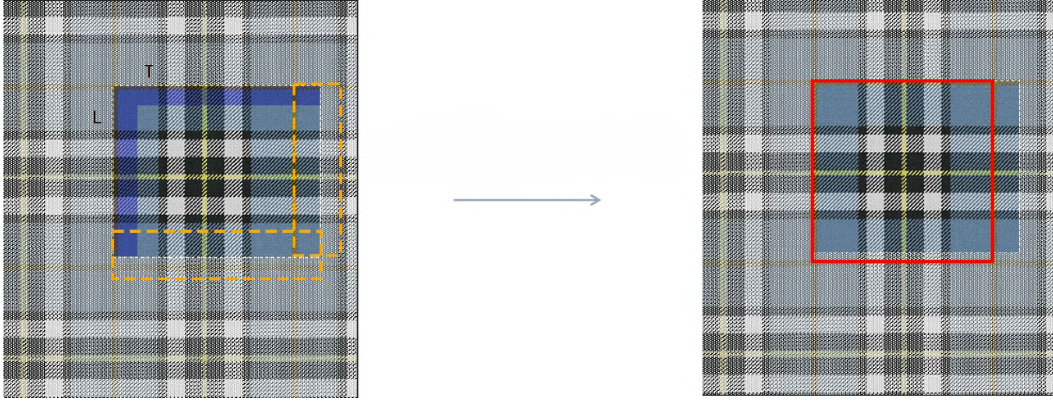


Fig. 5: Schema del processo di ricerca del modulo di ripetizione: selezione iniziale dell'utente (tratteggio bianco); bande di confronto interne T e L, evidenziate in blu, assunte come fisse; zona di ricerca definita attorno alla selezione (tratteggio arancione), all'interno della quale vengono individuate le bande esterne candidate B e R; modulo di ripetizione estratto (in rosso). Un modulo è considerato seamless quando le bande interne e le corrispondenti bande esterne risultano visivamente coincidenti.

con le corrispondenti **bande esterne** al modulo stesso, rispettivamente inferiore (B) e destra (R).

La ricerca delle bande esterne B e R avviene all'interno di una zona di ricerca definita attorno alla selezione iniziale. In tale regione, una finestra di ricerca viene fatta scorrere secondo un approccio a *sliding window*, generando diverse ipotesi di modulo ottenute tramite leggeri restringimenti o ampliamenti della selezione di partenza.

Per ciascuna posizione viene calcolato il valore della funzione di costo che confronta le due bande, basata sulla distanza quadratica media tra i pixel delle bande interne (T, L) e quelli delle corrispondenti bande esterne candidate (B, R), considerando separatamente i contributi orizzontale e verticale. La configurazione che minimizza la funzione di costo viene infine selezionata come modulo di ripetizione ottimale.

In particolare, la ricerca del modulo è formulata come un problema di minimizzazione della seguente funzione di costo (Eq. (1)):

$$\min_{\substack{t \in [-b, b] \\ s \in [-b, b]}} \left[\frac{1}{N} \sqrt{\sum_{i=1}^N \left(L_i - R_i^{(t)} \right)^2} + \frac{1}{M} \sqrt{\sum_{j=1}^M \left(T_j - B_j^{(s)} \right)^2} \right] \quad (1)$$

dove L_i e T_j rappresentano i pixel delle bande interne fisse, $R_i^{(t)}$ e $B_j^{(s)}$ i pixel delle bande esterne candidate dipendenti dallo spostamento della finestra di ricerca, N e M il numero di pixel delle bande verticale e orizzontale, e t , s gli spostamenti orizzontale e verticale all'interno dell'intervallo $[-b, b]$.

La procedura di ricerca del modulo viene eseguita considerando simultaneamente tutte le mappe del materiale fornite in ingresso. Ciascuna mappa (ad esempio albedo, normal, roughness) viene letta e rappresentata come una matrice di valori pixel; tali matrici vengono quindi combinate in una struttura dati multidimensionale che consente di trattare l'insieme delle mappe come un'unica immagine multicanale.

Prima della combinazione, ogni mappa viene opportunamente pesata in base al coefficiente fornito dall'utente (tramite il file di testo dedicato). Il peso associato a ciascuna

mappa modula il contributo dei suoi canali all'interno del confronto, consentendo di enfatizzare o attenuare l'influenza delle diverse proprietà del materiale nel processo di ricerca.

Il confronto tra le bande interne ed esterne, espresso dalla funzione di costo in Eq. (1), viene pertanto effettuato su tutti i canali disponibili, sommando i contributi pesati di tutte le mappe del materiale. In questo modo, la ricerca del modulo tiene conto in modo coerente e controllabile delle diverse proprietà fisiche rappresentate dalle mappe PBR.

L'algoritmo di ricerca opera in modo deterministico esplorando sistematicamente lo spazio delle configurazioni ammissibili all'interno della zona di ricerca. L'esplorazione avviene secondo una procedura sequenziale, in cui la posizione ottimale viene ricercata separatamente lungo le due direzioni principali, valutando dapprima una direzione e successivamente l'altra.

La scelta di un approccio basato sul confronto diretto dei bordi mediante una finestra di ricerca è motivata dalla natura delle texture considerate e dal contesto applicativo del software. Tale metodo consente di operare direttamente sulle immagini raster, senza richiedere ipotesi aggiuntive sulla periodicità globale del pattern. Questo metodo applicativo presuppone l'esistenza di un modulo di ripetizione e consente all'utente di fornire una selezione iniziale mirata, eventualmente escludendo porzioni dell'immagine affette da distorsioni o difetti locali.

3.1.1 Estensione al caso parallelogramma

Il metodo di ricerca del modulo può essere esteso al caso di selezioni di forma parallelogramma, al fine di gestire pattern caratterizzati da periodicità lungo assi non ortogonali (esempio riportato in Fig. 3). In questo scenario, l'utente fornisce una selezione iniziale inclinata, che definisce un sistema di riferimento affine rispetto al caso rettangolare.

Il procedimento di ricerca rimane invariato: anche in questo caso vengono confrontate bande interne ed esterne del modulo mediante una finestra di ricerca, e la selezione ottimale è individuata minimizzando la stessa funzione di costo. La differenza risiede esclusivamente nella gestione geometrica delle coordinate e nell'estrazione delle bande lungo direzioni inclinate, coerenti con la forma del parallelogramma selezionato.

3.1.2 Limitazioni

Il metodo di ricerca del modulo presenta alcune limitazioni intrinseche legate alle assunzioni alla base dell'algoritmo e alla natura delle texture considerate. In primo luogo, l'approccio presuppone l'esistenza di un modulo di ripetizione ben definito: nel caso di texture di tipo organico o altamente irregolari, in cui non è presente una periodicità riconducibile a un modulo ripetibile, l'algoritmo non è in grado di individuare una soluzione significativa.

Inoltre, la qualità del risultato può risentire di distorsioni introdotte durante il processo di digitalizzazione del tessuto (ad esempio deformazioni locali o variazioni di prospettiva), che compromettono la corrispondenza esatta tra i bordi del modulo e possono portare a un tiling non perfettamente seamless.

Infine, l'estensione del metodo al caso di selezioni a parallelogramma risulta sensibile all'inclinazione scelta dall'utente. Una selezione iniziale non sufficientemente allineata

alla reale periodicità del pattern può influenzare negativamente la ricerca, riducendo la qualità del modulo individuato.

3.2 Graph Cut

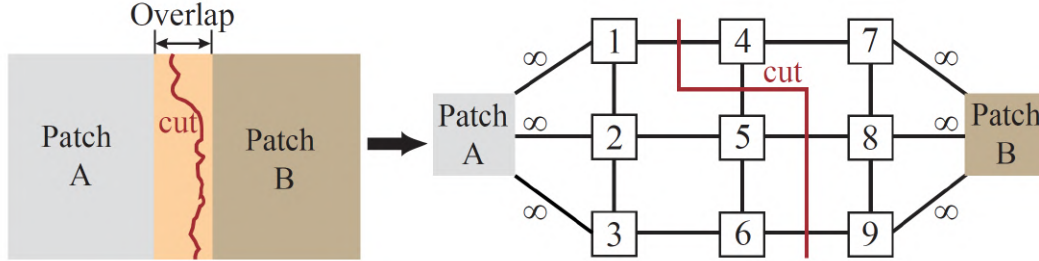


Fig. 6: Regione di sovrapposizione tra due patch (sinistra) e corrispondente modellazione del problema di individuazione della seam tramite Graph Cut (destra), con il taglio minimo evidenziato. Immagine da [2]

Le funzionalità di **subpatching** e di **seamless tiling avanzata** che sono state ideate si basano su un approccio di ottimizzazione tramite Graph Cut, ampiamente utilizzato in letteratura per problemi di sintesi e manipolazione di immagini. Il Graph Cut viene impiegato per individuare transizioni ottimali tra regioni adiacenti minimizzando la discontinuità visiva lungo i bordi di giunzione, senza imporre strutture rigide sul percorso di taglio, e permettendo di individuare sovrapposizioni complesse.

Nel contesto della sintesi o manipolazione di texture, il processo prevede la sovrapposizione di una nuova regione candidata (*patch*) a una porzione già presente nell'immagine di output (Fig. 6). L'obiettivo è determinare un percorso di separazione all'interno della regione di sovrapposizione che minimizzi la discontinuità visiva tra le due regioni, favorendo il passaggio del taglio attraverso aree in cui le differenze locali sono minime. Il problema viene quindi modellato come un grafo pesato, in cui i nodi rappresentano i pixel della regione di sovrapposizione e gli archi codificano un costo di discontinuità basato sulla differenza tra i valori dei pixel provenienti dalle diverse sorgenti. La soluzione ottimale è ottenuta tramite algoritmi di min-cut/max-flow, che garantiscono il calcolo efficiente del percorso di taglio a costo minimo.

L'esecuzione di un Graph Cut può essere schematizzata in due fasi principali:

1. l'individuazione di una patch candidata ottimale rispetto alla porzione di texture da modificare;
2. il calcolo del percorso di taglio a costo minimo all'interno della regione di sovrapposizione.

Scelta della patch

Data una regione della texture da modificare o correggere, viene innanzitutto individuata una patch candidata prelevata dalla texture stessa, che presenti caratteristiche

compatibili con la regione di destinazione. La selezione avviene valutando una funzione di similarità tra la patch candidata e la regione target.

In generale, la regione locale dell'immagine che si vuole modificare S_O viene confrontata con le traslazioni dell'immagine di input. Vengono considerate esclusivamente le traslazioni dell'immagine di input per le quali esiste una sovrapposizione con S_O . Per ciascuna traslazione valida t , viene valutata la similarità tra la patch candidata e S_O mediante una funzione di costo basata sulla somma pixel-wise delle differenze al quadrato:

$$C(t) = \sum_{p \in S_O} \|I(p - t) - O(p)\|^2, \quad (2)$$

dove I indica l'immagine di input e O l'output corrente. Valori ridotti della funzione di costo indicano una maggiore compatibilità tra la porzione di input considerata e la regione dell'output da correggere.

Scelta del taglio

Una volta individuata la patch candidata, la regione di sovrapposizione tra la patch e la texture esistente viene modellata come un grafo pesato $G = (V, E)$. Ogni nodo $v \in V$ rappresenta un pixel della regione di sovrapposizione, mentre gli archi $e \in E$ collegano pixel adiacenti e sono pesati in base al valore di discontinuità. In particolare, per la regione di sovrapposizione tra la patch esistente A e la nuova patch candidata B , si definisce, per ogni coppia di pixel adiacenti s, t , un costo di discontinuità tra le due patch dato da:

$$M(s, t, A, B) = \|A(s) - B(s)\| + \|A(t) - B(t)\|, \quad (3)$$

dove $\|\cdot\|$ indica una norma nello spazio colore (ad esempio la norma euclidea). Tale costo viene utilizzato come peso sugli archi del grafo che collegano i nodi corrispondenti a pixel adiacenti all'interno della regione di overlap.

Poiché i tagli risultano maggiormente visibili in regioni a bassa frequenza, queste possono essere penalizzate nella funzione (3). A tal fine viene definita una versione pesata della funzione di costo:

$$M'(s, t, A, B) = \frac{M(s, t, A, B)}{\|G_A^d(s)\| + \|G_A^d(t)\| + \|G_B^d(s)\| + \|G_B^d(t)\|}, \quad (4)$$

dove G_A^d e G_B^d rappresentano i gradienti delle patch A e B , rispettivamente, calcolati nella direzione d , nei punti s e t . Questa funzione di costo è descritta per una singola immagine e deve essere calcolata per ogni mappa del materiale in esame.

Per concludere la creazione del grafo, archi aggiuntivi di costo infinito vengono definiti in corrispondenza di alcuni pixel alle due patch (ad esempio le colonne o righe esterne della regione di overlap) per vincolare i bordi a provenire dalla patch vecchia o nuova. Così facendo si impedisce al taglio di attraversarli e quindi rimanere all'interno della regione di overlap.

Il problema della determinazione del percorso di taglio ottimale viene quindi ricondotto alla ricerca di un percorso minimo nel grafo costruito, che viene eseguito attraverso l'utilizzo di algoritmi di min-cut/max-flow.

3.2.1 Subpatching

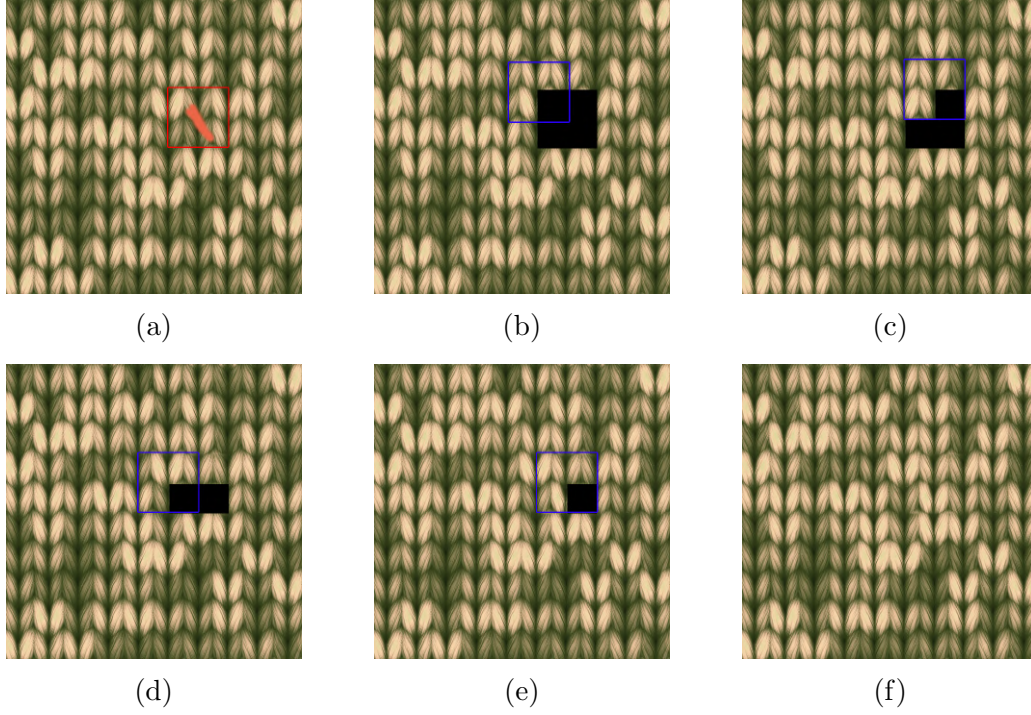


Fig. 7: Esempio di subpatching. (a) *Error region* selezionata dall'utente (rosso) contenente artefatti. (b)–(e) Aggiornamento iterativo della regione: l'area evidenziata in blu rappresenta la *error patch*, utilizzata come contesto per la ricerca della patch ottimale e come regione di sostituzione. (f) Texture risultante dopo il completamento del processo. La dimensione delle patch è pari a quella della selezione iniziale.

Il subpatching è una procedura finalizzata alla modifica locale di una texture, ideata per correggere regioni affette da artefatti o discontinuità visive. L'approccio adottato presuppone che l'area da correggere sia di estensione limitata rispetto all'immagine complessiva e che il contesto circostante fornisca informazioni sufficienti per una ricostruzione coerente.

L'utente seleziona interattivamente una regione dell'immagine, denominata *error region*, che identifica l'area da sostituire. Tale regione viene trattata come contenente informazione non affidabile e, pertanto, non viene utilizzata nel processo di selezione delle patch candidate, al fine di evitare la reintroduzione degli artefatti da correggere.

La procedura di subpatching è di tipo iterativo e ha inizio marcando come non validi tutti i pixel appartenenti alla *error region*. Successivamente, viene definita una patch di dimensione fissata, specificata dall'utente, che viene posizionata col centro sul primo pixel non valido della *error region* all'iterazione corrente. Questa regione locale viene indicata come *error patch*. Per ciascuna *error patch*, l'algoritmo ricerca nell'immagine originale una patch candidata di pari dimensioni, selezionata esclusivamente tra le traslazioni che garantiscono una copertura completa della *error patch*. Una volta individuata la patch candidata ottimale (come minimo di Eq. (2) su tutte le traslazioni t), si calcola il taglio ottimale tra le due patch. Al fine di introdurre una lieve variabilità e quindi ridurre la ripetitività visiva, la selezione della patch candidata non avviene

in modo strettamente deterministico. Invece di scegliere sempre la patch con costo minimo assoluto, l'algoritmo seleziona in modo stocastico una patch all'interno di un insieme di soluzioni il cui costo differisce dal minimo globale entro una soglia prefissata. La scelta finale viene quindi effettuata in modo casuale all'interno di questo insieme ristretto di patch al fine di rendere la texture risultante visivamente più naturale, senza comprometterne la coerenza.

Il processo viene ripetuto in modo iterativo (Fig. 7): a ogni iterazione, la porzione riempita diventa parte del contesto valido, e una nuova *error patch* viene definita in corrispondenza del successivo pixel non valido. La procedura termina quando l'intera *error region* risulta completamente sostituita.

Infine, il software STT prevede una modalità alternativa di fusione delle patch per mitigare casi in cui la patch ottimale e l'algoritmo di Graph Cut non riescono a garantire una transizione completamente priva di artefatti (ad esempio per la presenza di variazioni locali marcate o pattern complessi).

Nella modalità standard, la combinazione tra la patch esistente e quella nuova avviene in modo netto: i pixel situati su un lato della seam vengono assegnati alla patch precedente, mentre quelli sull'altro lato alla patch nuova, secondo il risultato del min-cut. In alternativa, l'utente può abilitare una modalità di fusione morbida, in cui il taglio viene trasformata in una zona di transizione graduale. A partire dalla maschera binaria che identifica le due regioni separate dal taglio, viene generata una maschera continua mediante l'applicazione di un filtro di sfocatura che viene quindi utilizzata per fondere progressivamente le due patch, producendo una transizione più morbida e meno percettibile.

L'implementazione del subpatching si basa su una rielaborazione e integrazione di un codice di riferimento disponibile pubblicamente su GitHub [5]. Il codice originale è stato adattato alle specifiche del software STT.

Le modifiche di subpatching vengono riportate su tutte le mappe del materiale in modo identico e coerente.

3.2.2 Seamless tiling avanzata

La procedura di seamless tiling avanzata consente di ottenere un modulo *tileable* anche in presenza di pattern di tipo organico. L'approccio adottato rappresenta una rielaborazione del metodo descritto in [3], a partire dal quale è stata sviluppata una soluzione adattata al contesto applicativo del software STT. L'implementazione originale di riferimento è resa disponibile dagli autori tramite un repository pubblico [6].

Il seamless tiling avanzata ha l'obiettivo di rendere perfettamente *tileable* il modulo individuato nella fase di ricerca del modulo, quando necessario. Per texture di tipo organico, infatti, il modulo ottimale trovato dal software STT minimizza le discontinuità visive ai bordi, ma non garantisce necessariamente una continuità perfetta tra lati opposti.

La procedura prende in ingresso il modulo ottimale individuato e un suo intorno esteso, ottenuto includendo una fascia di pixel attorno ai bordi del modulo. Tale intorno definisce una regione di sovrapposizione tra i lati opposti del modulo, all'interno della quale viene ricercata una transizione visivamente coerente.

Per ciascuna coppia di bordi opposti, il metodo individua innanzitutto una sovrapposizione ottimale tra le regioni adiacenti, valutando la compatibilità dei pixel lungo

l'area di overlap. Successivamente, viene costruito un grafo che modella il costo di discontinuità tra le due regioni sovrapposte (come descritto dall'equazione (4)) e viene risolto il problema di min-cut/max-flow per determinare un percorso di taglio a costo minimo. Il costo di discontinuità viene calcolato su tutte le mappe del materiale. Il taglio ottenuto definisce una cucitura irregolare lungo la quale le due regioni vengono combinate, garantendo una transizione senza artefatti visivi.

La porzione di bordo così ottenuta viene suddivisa e replicata su entrambi i lati opposti del modulo, rendendo il tiling continuo lungo quella direzione. La procedura viene applicata ai bordi sinistro e destro del modulo e ai bordi superiore e inferiore.

La fase di combinazione delle patch lungo il bordo segue un approccio analogo a quello adottato nel subpatching: invece di copiare rigidamente le regioni del bordo, il taglio viene utilizzato per costruire una maschera di fusione che permette una transizione graduale tra le patch.

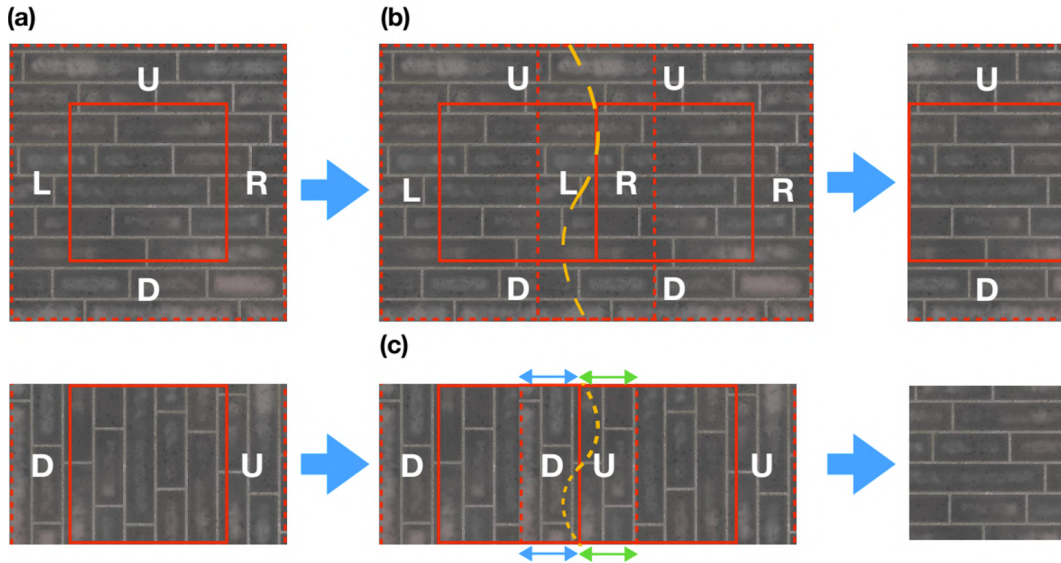


Fig. 8: Schema della procedura di *seamless tiling avanzata* basata su *graph cut*. (a) Modulo iniziale (in rosso) con bordo di estensione. (b) Modifica dei bordi sinistro (L) e destro (R) mediante individuazione di una regione di sovrapposizione e calcolo del taglio a costo minimo; (c) cucitura dei bordi superiore (U) e inferiore (D) con lo stesso procedimento. Il risultato finale è un modulo *tileable*, in cui le transizioni sui bordi opposti risultano visivamente continue. Immagine presa da [3]

3.2.3 Limitazioni dei metodi basati su Graph Cut

Le tecniche basate su Graph Cut presentano alcune limitazioni intrinseche, comuni alle due funzionalità implementate, nonché limitazioni specifiche legate ai singoli casi d'uso.

La prima limitazione comune alle due funzionalità è legata al costo computazionale dell'algoritmo di min-cut/max-flow, che nel caso peggiore presenta una complessità quadratica rispetto al numero di nodi del grafo [4].

Inoltre, entrambe le funzionalità basate su Graph Cut sono attualmente sviluppate esclusivamente per il caso di moduli di tiling rettangolari. L'estensione dei metodi al

caso di moduli a parallelogramma non è al momento supportata e può essere considerata come un possibile sviluppo futuro.

Limitazioni del Subpatching Nell’implementazione attuale, la selezione della patch candidata ottimale è effettuata considerando una singola mappa del materiale. Le modifiche risultanti vengono successivamente applicate in modo coerente a tutte le mappe PBR associate. Questa scelta rappresenta una limitazione implementativa, adottata per contenere la complessità computazionale, e non costituisce un vincolo intrinseco del metodo.

Il subpatching presenta alcune limitazioni legate alle caratteristiche della texture di partenza e ai parametri scelti dall’utente. Il metodo presuppone che l’immagine originale contenga un numero sufficiente di ripetizioni del modulo del tessuto, in modo da garantire la disponibilità di patch candidate adeguate; in caso contrario, la qualità del risultato può degradare.

Anche la dimensione della *error region* influisce sul risultato finale: regioni troppo piccole possono non fornire un contesto sufficientemente informativo per una selezione affidabile della patch ottimale. Analogamente, la dimensione della patch utilizzata nel subpatching rappresenta un parametro critico. Osservazioni empiriche indicano che risultati generalmente migliori si ottengono utilizzando patch di dimensione compresa tra 0.5 e 2 volte la dimensione del modulo di ripetizione, sebbene la scelta ottimale dipenda dalle caratteristiche specifiche della texture.

Limitazioni del Seamless Tiling avanzata La funzionalità di seamless tiling avanzata basata su Graph Cut è stata sviluppata e testata separatamente, ma non risulta attualmente integrata nel flusso operativo del software STT. Di conseguenza, tale metodo non è accessibile all’utente nella versione consegnata del software.

L’implementazione attuale della procedura di seamless tiling avanzata è scritta in linguaggio MATLAB, e non presenta una interfaccia grafica per l’utente.

La principale limitazione di questo metodo di generazione di moduli *seamless* è legata alla scelta dell’estensione del bordo attorno al modulo iniziale. In funzione delle caratteristiche strutturali del pattern della texture, può essere necessario adattare la dimensione di tale bordo affinché sia presente una la regione di sovrapposizione abbastanza ampia da contenere l’intera transizione dei due lati del modulo.

4 Modalità di utilizzo del software

4.1 Installazione STT

Il progetto richiede l’installazione di **Python** (si consiglia la versione 3.12).

Di seguito sono riportati i passaggi necessari per l’installazione e l’esecuzione del software STT. Tutti i comandi devono essere eseguiti dalla directory principale del progetto, e si userà il comando `python` per indicare la versione consigliata (per alcuni sistemi potrebbe essere necessario l’utilizzo di `python3`).

1. **Creazione e attivazione dell’ambiente virtuale.** Si consiglia l’utilizzo di un ambiente virtuale Python per evitare conflitti tra le dipendenze del progetto e altre librerie installate nel sistema. Ad esempio:

Linux / macOS

```
python3 -m venv venv
source venv/bin/activate
```

Windows

```
python -m venv venv
venv\Scripts\activate
```

2. **Installazione delle dipendenze.** Le dipendenze necessarie al funzionamento del software sono elencate nel file `requirements.txt`, situato nella directory principale del progetto. Con l'ambiente virtuale attivo, eseguire:

```
python -m pip install -r requirements.txt
```

Al termine dell'installazione, tutte le librerie richieste dal progetto saranno disponibili all'interno dell'ambiente virtuale.

3. **Esecuzione del progetto.** Eseguire l'applicativo con:

```
python main.py
```

4.2 Utilizzo del software STT

All'avvio, l'applicativo visualizza una texture di esempio. Nella barra dei menù sono presenti quattro voci principali: **File**, **Edit**, **Tiling** e **Selection**. Per caricare nuovi dati di input, il menù **File** offre due opzioni: **Open single image** e **Open image map in folder**. La prima consente di aprire una singola immagine, mentre la seconda permette di caricare un insieme di mappe PBR associate a un materiale.

Nel caso di caricamento di un materiale PBR, tutte le mappe devono essere contenute in una cartella (l'applicativo considera ogni immagine presente nella cartella di caricamento). L'utente seleziona una delle immagini del set tramite la finestra di dialogo del file system. Tale immagine viene visualizzata nell'interfaccia e utilizzata per l'interazione, mentre tutte le altre mappe del materiale vengono caricate internamente e successivamente modificate in modo coerente con essa.

L'applicazione fornisce strumenti di navigazione dell'immagine, includendo zoom e pan. Lo zoom può essere effettuato tramite la rotella del mouse (o lo scorrimento a due dita sul touchpad), mentre lo spostamento dell'immagine è possibile sia tramite le barre di scorrimento presenti ai bordi della finestra, sia mediante trascinamento diretto dell'immagine con una modalità di *click-and-drag* attivata dal tasto centrale del mouse (o dal tasto destro del touchpad).

Per avviare la ricerca del modulo di tiling, l'utente deve selezionare una regione della texture che approssimi il pattern di ripetizione. Di default, la modalità di selezione è

rettangolare; è tuttavia possibile passare alla modalità di selezione a parallelogramma (e viceversa) tramite le apposite voci del menù **Selection** → **Selection mode**.

La selezione dell'area di interesse avviene tramite una modalità di *click-and-drag* con il tasto sinistro del mouse. Il primo click definisce l'angolo superiore sinistro della selezione, mentre il trascinamento del cursore determina la posizione dell'angolo inferiore destro. L'area selezionata viene visualizzata mediante un contorno a tratteggio bianco.

La selezione di un'area a parallelogramma avviene in modo analogo: l'utente definisce inizialmente un rettangolo di base, che può successivamente essere inclinato. In particolare, ciascun lato del parallelogramma può essere modificato tramite *click-and-drag* con il tasto sinistro del mouse sul lato di interesse, in combinazione con il tasto **Shift** per lo spostamento lungo l'asse orizzontale o con il tasto **Ctrl** per lo spostamento lungo l'asse verticale.

Per annullare la selezione corrente, l'utente può effettuare un doppio click in qualsiasi punto dell'immagine.

Una volta definita l'area di interesse, la ricerca del modulo di tiling viene avviata selezionando la voce di menù **Tiling** → **Start tiling**. Al termine dell'elaborazione, il modulo individuato viene evidenziato sull'immagine tramite un contorno rosso e salvato automaticamente nella cartella di output.

A partire dal modulo trovato, è possibile visualizzare un'anteprima del tiling tramite la generazione di una disposizione 3×3 , attivabile dalla voce di menù **Tiling** → **Update image with tiled texture**.

In seguito a qualsiasi operazione, l'immagine visualizzata può essere ripristinata allo stato iniziale tramite la voce **Edit** → **Restore image**. In nessun momento dell'utilizzo del software i file di input originali vengono modificati o sovrascritti.

Per applicare la procedura di subpatching a una texture, l'utente deve selezionare un'area rettangolare dell'immagine e avviare l'operazione tramite la voce di menù **Edit** → **Subpatch**. Viene quindi visualizzata una finestra di dialogo in cui l'utente può specificare la dimensione delle patch, espressa come valore relativo alla dimensione dell'area selezionata (*error region*), e scegliere se utilizzare una modalità di fusione morbida delle patch (*seam blending*).

Al termine dell'esecuzione della procedura di subpatching, le modifiche apportate vengono visualizzate nell'immagine mostrata dall'applicazione e salvate automaticamente nella cartella di output.

4.3 Utilizzo del modulo di Seamless tiling avanzato

Per la guida su installazione ed utilizzo del modulo di Seamless tiling avanzato si rimanda alla documentazione della versione originale del codice [6].

Per eseguire il seamless tiling avanzato secondo il workflow previsto, il dato di ingresso della procedura è costituito dal modulo con bordo di estensione estratto dal software STT. Tale modulo viene salvato automaticamente nella cartella di output con il suffisso `_expansion_border`.

5 Risultati

In questa sezione vengono presentati alcuni risultati ottenuti mediante l'utilizzo del software STT, con particolare attenzione alla valutazione qualitativa delle texture generate.

5.1 Ricerca del modulo

In Fig. 9 sono mostrati alcuni risultati della procedura di estrazione del modulo di tiling. La prima riga riporta i moduli individuati dall'algoritmo, mentre la seconda riga mostra il corrispondente tiling 3×3 ottenuto a partire da ciascun modulo.

Per semplicità, vengono riportate esclusivamente le immagini relative alla mappa *albedo* dei materiali considerati. La ricerca del modulo rettangolare ha prodotto risultati soddisfacenti per i casi (a)–(d), sia nel caso di texture sintetiche caratterizzate da pattern periodici regolari (come nell'esempio (a)), sia per texture derivate dalla digitalizzazione di tessuti reali (ad esempio il modulo (c) e (d)).

I casi (d) e (e) rappresentano due esempi particolarmente significativi. L'esempio (d) mostra un caso di successo nella selezione di un modulo di tipo parallelogramma, correttamente estratto da una texture con pattern di ripetizione orientato lungo assi non ortogonali.

Infine, la texture (e) raffigura un asciugamano a spugna, caratterizzato da un pattern organico e privo di una struttura di ripetizione regolare. In questo caso, la ricerca del modulo risulta intrinsecamente problematica, poiché un modulo di ripetizione non è in effetti presente. Il modulo estratto corrisponde quindi alla soluzione che minimizza le discontinuità lungo i bordi di ripetizione. Rimangono però artefatti visivi evidenti, che possono essere ulteriormente attenuati mediante l'applicazione della procedura di seamless tiling avanzata.

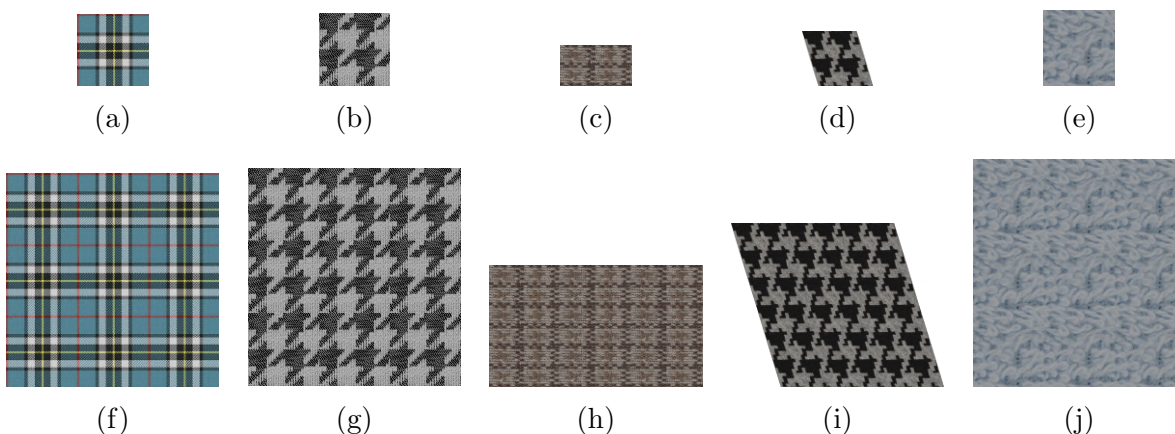


Fig. 9: Risultati della procedura di ricerca del modulo. Moduli estratti (prima riga) e tiling 3×3 del modulo stesso (seconda riga).

5.2 Subpatching

In Fig. 10 sono mostrati alcuni esempi di input e output della procedura di *subpatching* applicata a due diverse texture (anche in questo caso, per semplicità, sono riportate

unicamente le mappe *albedo*).

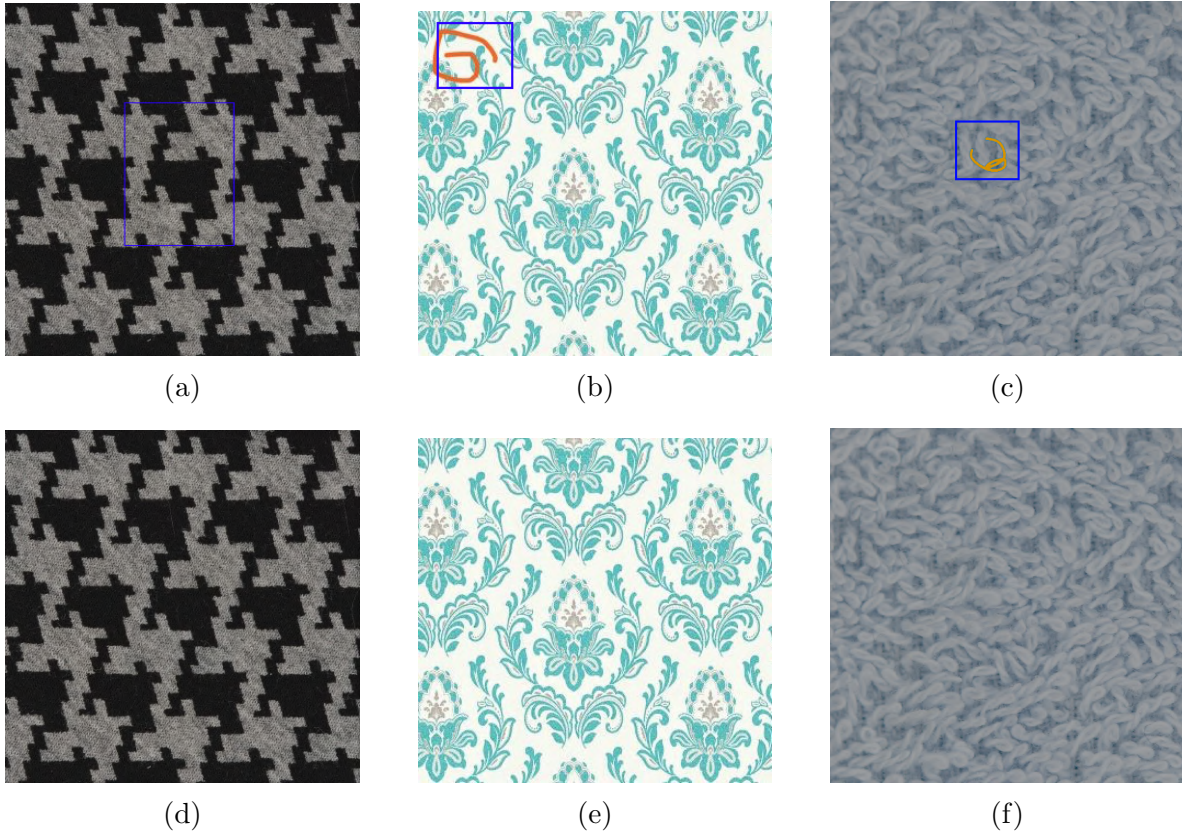


Fig. 10: Risultati della procedura di subpatching per due diverse texture. La prima riga riporta l'immagine di input con indicazione della regione di errore selezionata dall'utente (in blu). La seconda riga riporta l'output del subpatching.

Nel caso della texture (a), ottenuta dalla digitalizzazione di un tessuto reale, la procedura di subpatching consente di correggere efficacemente l'area selezionata. In questo esempio, la dimensione della patch è stata impostata uguale a quella della *error region*, risultando leggermente superiore alla dimensione del minimo elemento di ripetizione del pattern. La scelta appropriata della dimensione della regione di errore, combinata con una dimensione di patch adeguata e con la presenza di un numero sufficiente di ripetizioni del modulo nell'immagine di input, permette di ottenere un risultato visivamente coerente e privo di artefatti, come mostrato nell'output (d).

Analogamente, la texture (b), che presenta un artefatto localizzato nell'angolo superiore sinistro (una spirale arancione sovrapposta al pattern), viene correttamente modificata dalla procedura di subpatching (e). In questo caso, l'algoritmo riesce a rimuovere l'artefatto preservando la continuità visiva della texture, dimostrando l'efficacia del metodo anche in presenza di disturbi grafici localizzati.

Analogamente, nell'ultimo esempio relativo alla texture dell'asciugamano (c), l'artefatto presente nella regione di errore selezionata dall'utente viene rimosso con successo dal subpatching, dimostrando l'efficacia del metodo anche su texture di tipo organico.

Ulteriori esempi sono riportati in Fig. 11, con l'obiettivo di evidenziare l'influenza della scelta dei parametri di esecuzione del subpatching applicato alla stessa texture. La texture considerata presenta un pattern geometricamente regolare, ma caratterizzato

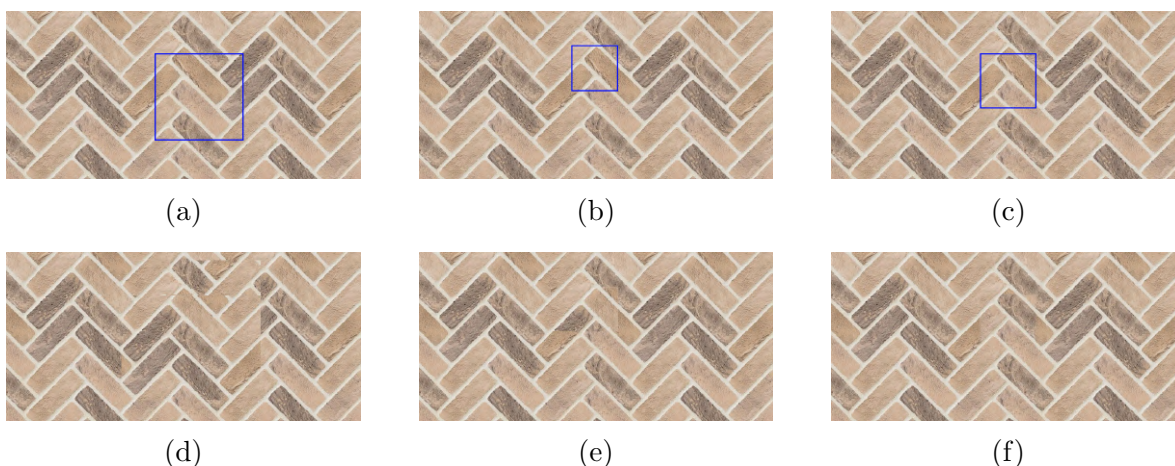


Fig. 11: Risultati del *subpatching* sulla stessa texture con diversi parametri di esecuzione. La prima riga mostra le immagini di input con la *error region* evidenziata in blu, mentre la seconda riga riporta gli output della procedura.

da variazioni cromatiche evidenti. In particolare, pur essendo presenti numerose ripetizioni dei singoli elementi (mattoni), non esiste una ripetizione esatta della specifica combinazione di mattoni situata nella regione centrale dell'immagine.

Nell'esempio (a) viene selezionata una *error region* relativamente estesa, che comprende questa combinazione unica di mattoni. Il subpatching viene eseguito utilizzando una dimensione della patch pari a $\frac{4}{5}$ di quella della *error region* e disabilitando la modalità di *seam blending*. Come visibile nel risultato (d), la procedura non riesce a produrre una modifica completamente *seamless*: in particolare, nella parte superiore dell'area modificata il pattern geometrico non viene ricostruito correttamente. Questo comportamento è riconducibile alla combinazione dei parametri scelti, ovvero una *error region* troppo estesa associata a una dimensione di patch elevata, in un contesto in cui la texture non contiene un numero sufficiente di ripetizioni dell'area di interesse.

Mantenendo la selezione in questa zona unica della texture, ma riducendo la dimensione della *error region*, come mostrato nell'esempio (b), e impostando una dimensione della patch uguale a quella della regione di errore, il subpatching riesce invece a ricostruire correttamente la struttura geometrica del pattern. Nell'output corrispondente (e), in cui la modalità di *seam blending* è ancora disattivata, risulta evidente una transizione netta tra le patch combinate, dovuta alle variazioni di colore della texture.

Per attenuare questo effetto visivo è possibile abilitare la modalità di fusione morbida delle patch (*seam blending*). Con una selezione simile dell'area di errore, come nel caso (c), il risultato finale (f) mostra una transizione più graduale tra le patch, producendo una modifica visivamente più coerente e realistica.

5.3 Seamless tiling avanzato

La Fig. 12 riporta alcuni risultati della procedura di seamless tiling avanzata applicata a due differenti texture, messi a confronto con i risultati ottenuti a partire dai moduli estratti dal software STT (con opportuno bordo di espansione).

Nella prima riga è mostrato il comportamento dell'algoritmo nel caso di texture di tipo organico, come l'asciugamano. A partire dalla texture originale viene estratto tra-

mite STT il modulo (a), che produce un tiling caratterizzato dalla presenza di artefatti sui bordi di ripetizione: in particolare, i fili del tessuto risultano interrotti lungo le giunzioni. L'output della procedura di seamless tiling avanzata è riportato nel modulo modificato (c), dal quale si ottiene il tiling (d), che risulta privo di discontinuità visive e quindi perfettamente seamless.

Un comportamento analogo si osserva nel secondo esempio. Il modulo (e), estratto da STT, genera un tiling (f) continuo dal punto di vista geometrico, ma presenta artefatti sui bordi di ripetizione dovuti alle variazioni di colore degli elementi del pattern (i mattoni). Queste discontinuità vengono efficacemente eliminate dalla procedura di seamless tiling avanzata: il modulo modificato (g) consente di ottenere il tiling (h), visivamente continuo. In questo caso l'algoritmo interviene esclusivamente sulla colorazione del modulo (tramite Graph Cut), senza modificarne le dimensioni.

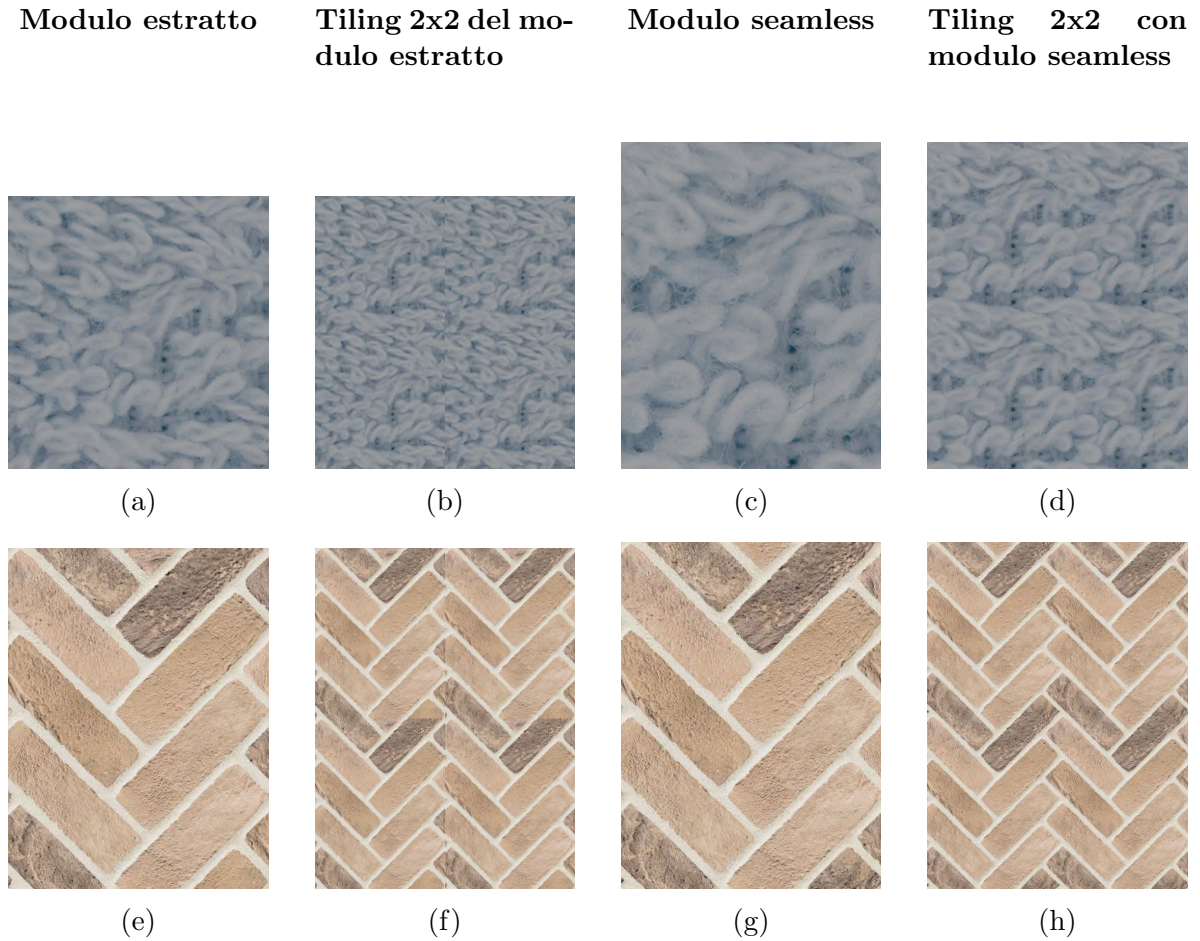


Fig. 12: Esempi di seamless tiling avanzato per due differenti texture. Nelle prime due colonne sono mostrati il modulo estratto dal software STT e il corrispondente tiling 2×2 . La terza e la quarta colonna riportano, rispettivamente, il modulo ottenuto mediante la procedura di seamless tiling avanzato e il relativo tiling 2×2 .

Dall'analisi dei due esempi precedenti emerge come la dimensione del modulo venga modificata dalla procedura di seamless tiling avanzata solo nel caso di pattern completamente organici. Nel secondo esempio, infatti, sebbene la colorazione dei mattoni non sia uniforme, la fase di ricerca del modulo eseguita dal software STT è in grado

di individuare correttamente la periodicità del pattern geometrico, e le modifiche del seamless tiling preservando le dimensioni del modulo originale.

La Fig. 13 riporta invece un caso in cui la procedura di seamless tiling avanzata non riesce a individuare un overlap corretto. Nella prima colonna sono mostrati il modulo (a) estratto da STT e il relativo tiling 2×2 (d). Il modulo ricostruisce correttamente la struttura geometrica della maglia, ma sono presenti artefatti visivi lungo la giunzione orizzontale, riconducibili a lievi variazioni di forma e di colore dei singoli elementi del pattern.

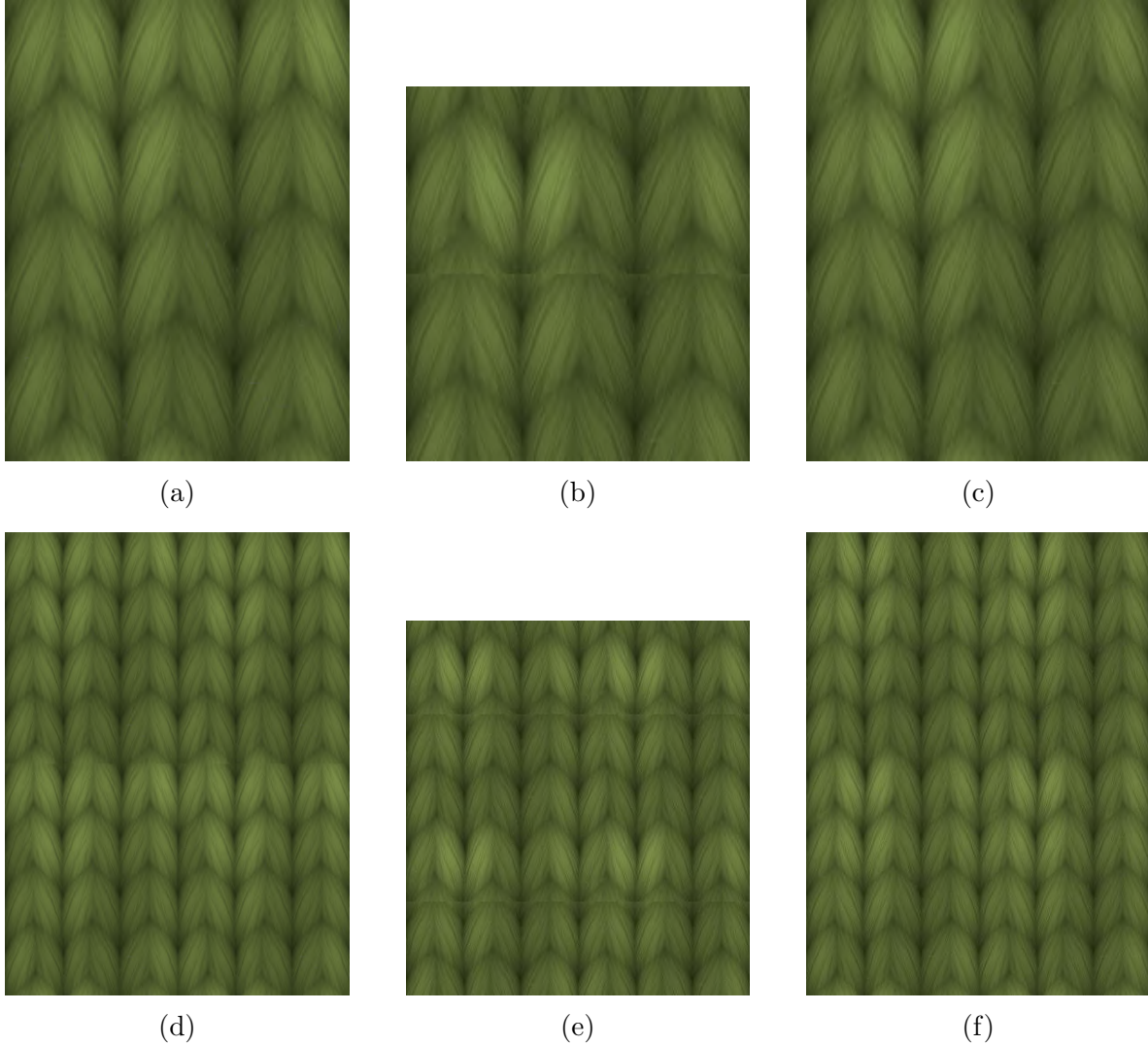


Fig. 13: Tre risultati di tiling della stessa texture a confronto. Sulla prima riga è riportato l'output del software STT; la seconda e la terza colonna riportano due diverse soluzioni della procedura di seamless tiling avanzata. Il modulo di output (b) è una modifica errata dell'input (a), come visibile dal tiling (e). La soluzione corretta è ottenuta dal modulo (c) come visibile dal suo tiling (f).

Utilizzando come input il modulo (a) con l'aggiunta del bordo di espansione, la procedura di seamless tiling restituisce il modulo (b), che risulta evidentemente scorretto. Tale comportamento è attribuibile alla fase di ricerca automatica della dimensione ot-

timale del bordo di sovrapposizione in cui calcolare il Graph Cut, che in questo caso non risulta sufficientemente robusta rispetto a variazioni cromatiche di bassa entità.

Tuttavia, questo specifico problema può essere risolto eliminando la ricerca automatica della dimensione dell'overlap e calcolando il graph cut direttamente in una regione di sovrapposizione di dimensioni pari al bordo di espansione aggiunto al modulo estratto da STT. Il bordo di espansione rimane necessario per consentire al graph cut di creare una transizione corretta tra i lati opposti del modulo, ma imponendo una sovrapposizione fissata è possibile ottenere un bordo seamless senza alterare le dimensioni del modulo originale.

6 Conclusioni

Il presente lavoro ha descritto lo sviluppo del software STT (Seamless Tiling Tool), progettato per la generazione automatica di texture seamless a partire da immagini PBR di tessuti digitalizzati. L'obiettivo del progetto era la realizzazione di uno strumento in grado di supportare l'individuazione e la costruzione di moduli di ripetizione coerenti, operando simultaneamente su tutte le mappe PBR associate al materiale e garantendo la consistenza delle proprietà fisiche durante l'intero processo di elaborazione.

La procedura di ricerca del modulo implementata in STT si è dimostrata efficace nel caso di texture caratterizzate da pattern periodici ben definiti, sia di origine sintetica sia derivanti dalla digitalizzazione di tessuti reali. In tali condizioni, il software è in grado di individuare moduli di ripetizione corretti e di generare tiling privi di discontinuità evidenti. Nel caso di texture di tipo organico, per le quali un modulo di ripetizione rigoroso non è intrinsecamente definibile, l'algoritmo restituisce soluzioni ottimali in termini di minimizzazione delle discontinuità sui bordi, senza garantire un risultato perfettamente seamless.

Per gestire tali casi, sono state sviluppate e testate funzionalità avanzate basate su Graph Cut, in particolare le procedure di *subpatching* e di *seamless tiling avanzato*. Il subpatching consente la correzione locale di artefatti all'interno delle texture, mentre il seamless tiling avanzato permette di ottenere moduli pienamente tileable anche in presenza di pattern organici, a condizione di una selezione appropriata dei parametri di esecuzione. I risultati sperimentali mostrano come tali approcci siano efficaci, ma fortemente dipendenti dalle caratteristiche della texture e dalla configurazione dei parametri.

L'analisi dei risultati ha inoltre evidenziato alcune limitazioni dell'attuale implementazione. In particolare, gli algoritmi basati su Graph Cut presentano un costo computazionale elevato e una sensibilità significativa alla scelta di parametri quali la dimensione delle patch, dell'area di errore e dell'overlap. In particolare, l'implementazione Graph Cut presente STT utilizza una libreria di min-cut/max-flow non ottimizzata. I tempi di esecuzioni sono quindi penalizzati rispetto all'implementazione utilizzata nel codice (ancora non integrato in STT) del seamless tiling avanzato. L'adozione di una implementazione ottimizzata del min-cut/max-flow migliorerebbe i tempi di esecuzione del subpatching. Inoltre, tali funzionalità sono attualmente limitate al caso di moduli rettangolari e la procedura di seamless tiling avanzato non risulta ancora integrata nel workflow principale del software STT.

Dal punto di vista computazionale, l'implementazione attuale dell'algoritmo di ricerca del modulo opera in modo sequenziale, esplorando sistematicamente lo spazio delle configurazioni mediante cicli annidati. Sebbene ciò garantisca un comportamento deterministico e facilmente controllabile, comporta tempi di esecuzione elevati per immagini ad alta risoluzione o per ampie zone di ricerca. Un possibile sviluppo futuro riguarda la parallelizzazione di alcune fasi del processo, in particolare la valutazione della funzione di costo per configurazioni indipendenti.

Gli sviluppi futuri suggeriti per il progetto includono l'integrazione completa del seamless tiling avanzato all'interno dell'applicativo, l'estensione dei metodi basati su Graph Cut al caso di moduli parallelogrammi e l'ottimizzazione delle prestazioni mediante la parallelizzazione delle fasi computazionalmente più onerose. Ulteriori miglioramento riguarda il consolidamento dell'interfaccia grafica, al fine di rendere il software più robusto e maggiormente adatto a contesti applicativi reali, così come l'aggiunta di parametri configurabili da un utente avanzato (ad esempio, la quantità di sfocatura nel *seam blending* e l'imposizione di un bordo a dimensione fissa nel seamless tiling avanzato). Sarà necessario un approfondito testing per controllare la correttezza delle funzionalità dell'interfaccia grafica e degli algoritmi.

Riferimenti bibliografici

- [1] L. Ford, D. Fulkerson, R. Bland, and J. Orlin. *Flows in Networks*. Princeton Landmarks in Mathematics and Physics. Princeton University Press, 2024.
- [2] V. Kwatra, A. Schödl, I. Essa, G. Turk, and A. Bobick. Graphcut textures: Image and video synthesis using graph cuts. *Acm transactions on graphics (tog)*, 22(3):277–286, 2003.
- [3] Z. Li, M. Shafiei, R. Ramamoorthi, K. Sunkavalli, and M. Chandraker. Inverse rendering for complex indoor scenes: Shape, spatially-varying lighting and svbrdf from a single image. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2475–2484, 2020.
- [4] R. Sedgewick. *Algorithms in C++ Part 5: Graph Algorithms*. Pearson Education, 2001.
- [5] Suffoquer-fang. Graph Cut <https://github.com/Suffoquer-fang/GraphCut/tree/main>, 2022. [Online; accessed 14-01-2026].
- [6] L. Zhengqin. Tileable Texture Synthesis <https://github.com/lzqsd/TileableTextureSynthesis/tree/master>, 2020. [Online; accessed 14-01-2026].